

PYTHON FOR DATA SCIENCE • MODULE 1 • TOPIC 1.14

pandas: Series & DataFrames

Day 15 — NumPy underneath, names on top

90 min

Concepts + live code

30 min

Practice notebook

pd

The import from here on

Prepared & presented by **Srinivasa Sai Chava**



Boston University

Before we start — five questions from Day 14

1 Mean or median for skewed data?

→ Median. One outlier drags the mean.

2 Why does `d == np.nan` find nothing?

→ `nan` is not equal to itself. Use `np.isnan`.

3 What does one `nan` do to `d.mean()`?

→ Makes the whole result `nan`.

4 `np.nansum` of an all-missing column?

→ 0.0 — indistinguishable from real zeros.

5 `corrcoef` gives you what?

→ A matrix. Take `[0, 1]`.

Question 3 has a surprising answer in pandas. Hold on to it — we will come back to it in section 2.

How today's two hours are structured

1

Why pandas

NumPy plus labels

10 min

2

Series

A labelled 1-D array — and a surprise

25 min

3

DataFrame

Creating, columns, single vs double brackets

30 min

4

Inspecting data

head, info, describe, value_counts, read_csv

20 min

5

Mini build

Load a CSV, add columns, summarise

5 min

6

Practice notebook

You code, I walk around

30 min

Tomorrow covers selecting rows properly with loc and iloc. Today is about the objects themselves.

By the end of today you will be able to

1

Build

Create a Series and a DataFrame from lists, dicts, and a CSV file

2

Select

Pull out one column or several, and know which object you got back

3

Inspect

Get the shape, types and missing counts of an unfamiliar dataset

4

Derive

Add new columns computed from existing ones, with no loops

Why this is the most important library in the course

Every dataset from here to the capstone will arrive as a DataFrame. Cleaning, exploring, feeding a model, reporting results — all of it happens in pandas. Time spent getting fluent now is repaid every single session for the next three and a half months.

1

Why pandas

NumPy plus labels · 10 minutes

- What NumPy lacks
- The one-sentence idea

What NumPy cannot do for you

A NumPy array has no names

```
marks = np.array([[88, 71],  
                  [91, 84],  
                  [45, 38]])  
  
marks[1, 0]    # which student? which subject?
```

- Rows and columns are just positions
- One type per array — no mixing names with numbers

A DataFrame names everything

```
df = pd.DataFrame({  
    "name":    ["Ravi", "Sara", "Amit"],  
    "python": [88, 91, 45],  
    "stats":  [71, 84, 38]})  
df["python"]  # obvious
```

- Columns have names; rows have labels
- Each COLUMN can have its own type

The whole idea in one sentence

A DataFrame is a collection of NumPy arrays — one per column — with names attached to the columns and labels attached to the rows. Everything from Days 12 to 14 still works underneath: broadcasting, masks, the axis rule. pandas adds the names, the mixed types, and a great deal of convenience on top.

2

Series

A labelled 1-D array · 25 minutes

- values + index
- Custom labels
- Vectorised, as before
- The nan surprise

A Series is a NumPy array with labels

Default labels: 0, 1, 2 ...

```
s = pd.Series([88, 71, 64])
```

```
0    88    <- index | values
1    71
2    64
dtype: int64
```

```
s.values    -> array([88, 71, 64])    NumPy!
s.index     -> RangeIndex(0, 3)
```

Your own labels

```
s = pd.Series([88, 71, 64],
              index=["Ravi", "Sara", "Amit"])
```

```
s["Sara"]    -> 71
```

```
pd.Series({"Ravi": 88, "Sara": 71})
# from a dict: keys become the index
```

It behaves like Day 5's dictionary AND Day 12's array

```
s["Sara"]    look up by label    - like a dict
s * 2        vectorised          - like an array
s[s > 70]    a boolean mask      - like an array
```

That combination is the reason pandas exists. You get named lookup and fast whole-column arithmetic in the same object, which neither a dict nor an array could give you on its own.

The surprise — pandas skips nan by default

The same four numbers, two libraries

```
arr = np.array([88., 71., np.nan, 64.])  
  
arr.mean()           -> nan  
pd.Series(arr).mean() -> 74.33  
  
pd.Series(arr).mean(skipna=False) -> nan
```

	NumPy	pandas
nan in a mean	returns nan	skips it
std default	ddof=0	ddof=1
to change it	np.nanmean	skipna=False

Why pandas made this choice

pandas was built for real tables, where missing values are normal rather than exceptional. Returning nan for every column with one gap would make it almost useless on real data — so it skips them, and assumes you have already looked.

⚠ Convenient — and quietly dangerous

NumPy's loud nan told you something was missing. pandas stays silent, so a mean over 3 of 4 values looks exactly like a mean over all 4. Yesterday's rule matters MORE in pandas, not less: always check how many values a summary actually used.

3

DataFrame

The labelled table · 30 minutes

- Creating one
- Anatomy
- Selecting columns
- One bracket or two

Three ways to build a DataFrame

1. A dict of lists — one list per COLUMN

```
df = pd.DataFrame({  
    "name": ["Ravi", "Sara", "Amit"],  
    "python": [88, 91, 45],  
    "stats": [71, 84, 38],  
})
```

The most common way. Every list must be the same length.

2. A list of dicts — one dict per ROW

```
rows = [  
    {"name": "Ravi", "python": 88},  
    {"name": "Sara", "python": 91},  
]  
df = pd.DataFrame(rows)
```

Natural when rows arrive one at a time — Day 9's DictReader gave you exactly this.

3. From a file — the one you will use most

```
df = pd.read_csv("marks.csv")  
  
# handles quoted commas (Day 9)  
# turns blank cells into NaN (Day 14)
```

Remember the Day 9 promise: one line to replace everything you did by hand with `csv.reader`. It reads the header, parses the quoting correctly, guesses a type for every column, and marks blank cells as missing. Tomorrow's sessions all begin with this line.

The anatomy of a DataFrame

the COLUMNS ↓

	name	city	python	stats
0	Ravi	Pune	88	71
1	Sara	Mumbai	91	84
2	Amit	Delhi	45	38
3	Neha	Pune	67	73

↑ the INDEX

Ask it about itself

```
df.shape      -> (4, 4)
df.columns    -> ['name', 'city',
                  'python', 'stats']
df.index      -> RangeIndex(0, 4)

df.dtypes
name         object   (str in pandas 3)
python       int64
```

Each column has its OWN type. That is the difference from a NumPy array, which forces one type on everything. Here name is text and python is an integer, side by side. Text columns show as `object` in pandas 2 and `str` in pandas 3 — you may see either depending on your Colab version, and they mean the same thing.

One bracket or two — and what comes back

One column → a Series

```
df["python"]  
  
0    88  
1    91  
2    45  
type: Series
```

Single brackets, a single string. You get the column as a Series — with its index still attached.

Several columns → a DataFrame

```
df[["name", "python"]]  
  
   name  python  
0  Ravi     88  
1  Sara     91  
type: DataFrame
```

DOUBLE brackets: the outer pair selects, the inner pair is a Python list of names.

⚠ Forgetting the inner brackets

```
df["name", "python"]  
-> KeyError  
# Python reads this as ONE key: a tuple
```

⚠ df.column works — until it does not

```
df.python    fine  
df.count     a METHOD, not your column!  
df.my score  SyntaxError - the space
```

Adding columns — vectorised, exactly like NumPy

```
df["total"] = df["python"] + df["stats"]  
df["average"] = df["total"] / 2  
df["passed"] = df["total"] >= 100
```

	name	python	stats	total	average	passed
0	Ravi	88	71	159	79.5	True
1	Sara	91	84	175	87.5	True
2	Amit	45	38	83	41.5	False
3	Neha	67	73	140	70.0	True

Nothing new here

Column plus column is Day 13's element-wise arithmetic. Dividing by 2 is broadcasting a scalar. The `>=` comparison produces a boolean column, exactly like a Day 12 mask.

Assigning to a name that does not exist creates the column; assigning to one that does replaces it.

Never loop over the rows to build a column

Writing `for i in range(len(df))` to fill a column is the most common pandas mistake there is. It works, and on real data it is hundreds of times slower than the one-line version above. If you find yourself looping over a DataFrame, stop and ask what the whole-column expression would be — the same rule as Day 13.

4

Inspecting Data

The first five minutes with any dataset · 20 minutes

- head & tail
- info
- describe
- value_counts

The first five commands on any new dataset

Command	Tells you	Why it matters
<code>df.head()</code>	the first 5 rows	what the data actually looks like
<code>df.shape</code>	(rows, columns)	is it the size you expected?
<code>df.info()</code>	types + non-null counts	the fastest way to spot missing data
<code>df.describe()</code>	count, mean, std, quartiles	Day 14's summary, every column at once
<code>df["col"].value_counts()</code>	how often each value appears	check a text column for typos

`info()` — read the Non-Null column

```
# Column Non-Null Count Dtype
2 mark 2 non-null float64 <- 1 missing
```

`describe()` — Day 14, all at once

Its count row is the number of NON-missing values, and its std uses `ddof=1`. Compare count with `len(df)` to see what is missing.

Debug corner — eight pandas bugs

You wrote	What happens	Fix
<code>df["name", "python"]</code>	KeyError — read as one tuple	<code>df[["name", "python"]]</code>
<code>df.count</code>	the method, not your column	<code>df["count"]</code>
<code>df["Python"]</code>	KeyError — names are case-sensitive	check <code>df.columns</code>
<code>s.mean()</code> expecting nan	pandas skips missing silently	check the count first
<code>arr.std()</code> vs <code>s.std()</code>	different answers — ddof 0 vs 1	state ddof explicitly
<code>for i in range(len(df))</code>	works, hundreds of times slower	a whole-column expression
<code>pd.DataFrame({..})</code> unequal lists	ValueError: must be same length	check every list
<code>int</code> column shows 88.0	a NaN forced it to float	<code>df["col"].isna().sum()</code>

Row 4 is the one to remember — pandas hides missing values that NumPy would have shouted about. All eight are in today's notebook.

Putting it together — from a CSV to a report

```
import pandas as pd
import numpy as np

df = pd.read_csv("marks.csv")          # one line, Day 9's promise

# ---- 1. inspect before trusting anything
print(df.shape)
print(df.head())
print(df.isna().sum())                # missing per column

# ---- 2. derive, without a loop
df["total"] = df["python"] + df["stats"]
df["average"] = df["total"] / 2
df["result"] = np.where(df["average"] >= 50,
                        "pass", "fail") # Day 13

# ---- 3. summarise, honestly
print(df[["python", "stats", "average"]].describe().round(1))
print(df["result"].value_counts())
print(df["city"].value_counts())

# ---- 4. save a NEW file - never overwrite the input
df.to_csv("marks_report.csv", index=False)
```

**read_csv**

Day 9's whole session, in one line

**Inspect first**

shape, head, missing counts

**Vectorised columns**

Day 13, with names on

**np.where**

Day 13's vectorised if/else

**describe()**

Day 14's summary, every column

**value_counts()**

Day 5's counting pattern

**index=False**

a NEW file, no stray index column

Every tool here is one you already know — pandas just lets you use them all on a named table at once.

Practice session — 30 minutes



Predict the output

Series, brackets, nan, dtypes

8 questions



Spot & fix the bug

Tuple key, df.count, loops, case

4 questions



Write the code

Build, select, derive, inspect, count

5 tasks

Your notebook

Day15_Practice_Questions.ipynb

- Run head() and info() first
- Series or DataFrame? check type()
- Raise a hand — I am walking around
- Solutions shared after class

Before any calculation, look at the data. head(), shape and info() take ten seconds and prevent most of today's bugs.

TODAY IN TWELVE LINES

Recap

- 1 import pandas as pd — always that alias.
- 2 A Series is a NumPy array with labels on it.
- 3 A DataFrame is NumPy columns with names on them.
- 4 Each column has its own type; an array has only one.
- 5 pandas SKIPS nan by default. NumPy does not.
- 6 pandas std uses ddof=1; NumPy uses ddof=0.
- 7 df["col"] gives a Series; df[["a", "b"]] a DataFrame.
- 8 Prefer df["col"] over df.col — count and sum are methods.
- 9 New columns are whole-column expressions. Never loop.
- 10 Inspect first: head, shape, info, describe.
- 11 info() shows the non-null count — missing data at a glance.
- 12 to_csv(..., index=False) to avoid a stray index column.

Homework

Rebuild Day 9's CSV report in pandas and compare the line counts · Create a DataFrame of 5 things you own, with 4 columns of different types · Run the five inspection commands on any CSV you can find

Next class — Topic 1.15

Filtering, selection & indexing: loc versus iloc, boolean filtering on a DataFrame, and setting a meaningful index